TASK1:

PLAYER WINS SCREENSHOT

```
Week4TUTsoftwareTech.py
```

```
        selected = moves[moves_keys[enemy_input]]  # Get element at index
        print("Enemy selected: ", moves_keys[enemy_input])

        enemy_health += selected[0]
        player_health += selected[1]
```

```
PLAYER TURN
Select move...
last stand
You selected:  last stand
Player health: 30
Enemy health: 5

ENEMY TURN
Enemy selected:  normal
Player health: 10
Enemy health: 5

PLAYER TURN
Select move...
normal
Invalid move. Try again.
Select move...
normal
You selected:  normal
Player health: 10
Enemy health: -15

GAME OVER: Player wins.

Process finished with exit code 0
```

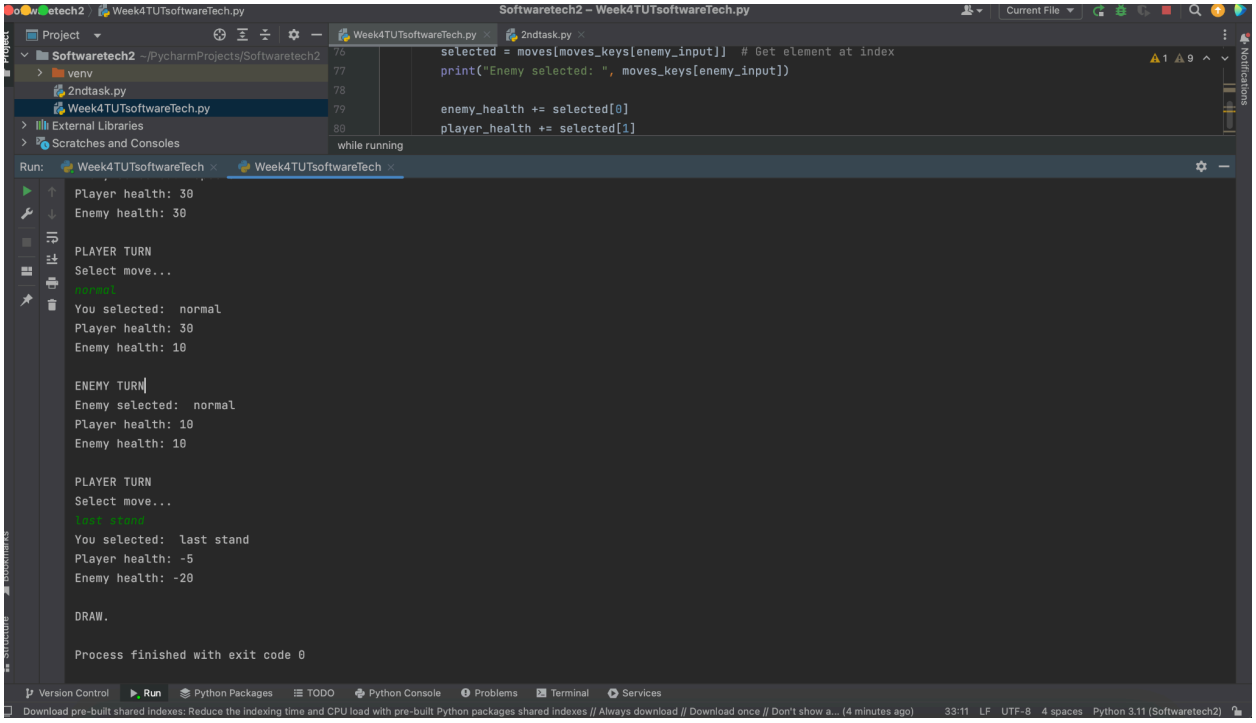
ENEMY WINS SCREENSHOT

```
Week4TUTsoftwareTech.py
```

```
        selected = moves[moves_keys[enemy_input]]  # Get element at index
        print("Enemy selected: ", moves_keys[enemy_input])

        enemy_health += selected[0]
        player_health += selected[1]
```

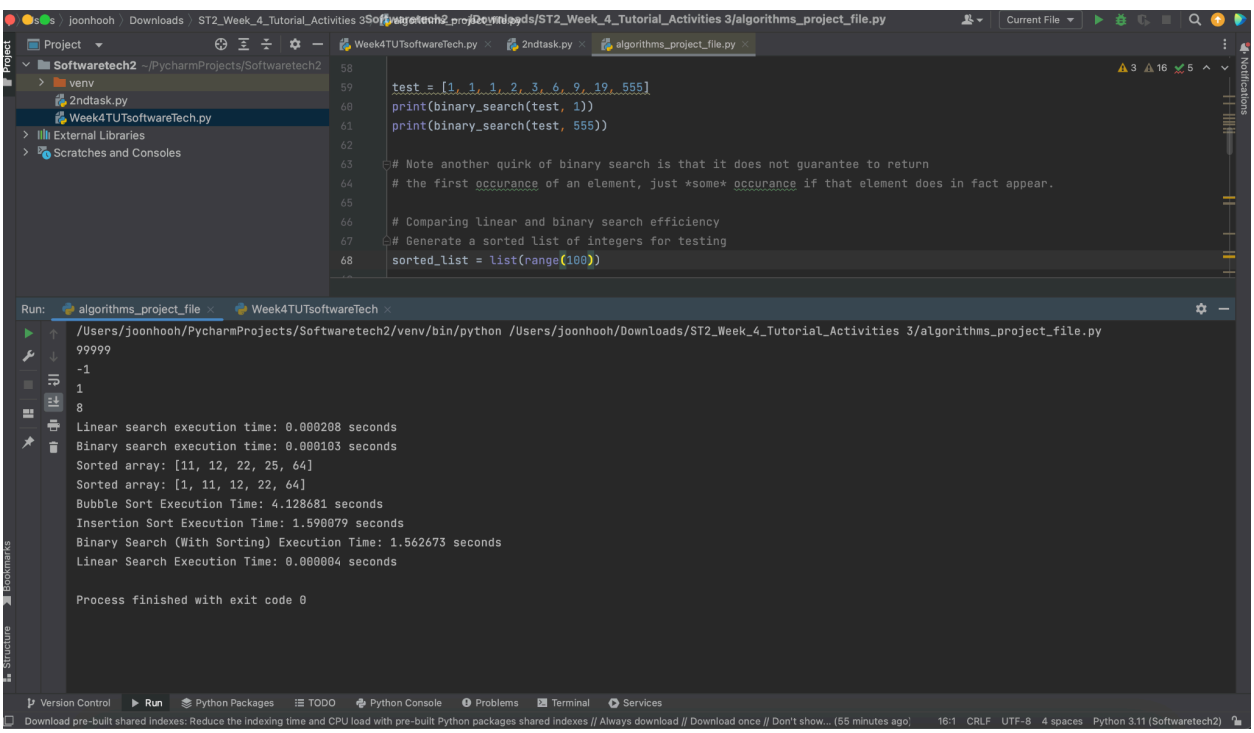
```
special
You selected:  special
Player health: 25
Enemy health: 20

ENEMY TURN
Enemy selected:  normal
Player health: 5
Enemy health: 20

PLAYER TURN
Select move...
special
You selected:  special
Player health: 10
Enemy health: 10

ENEMY TURN
Enemy selected:  special
Player health: 0
Enemy health: 15

GAME OVER: Enemy wins.

Process finished with exit code 0
```

DRAW SCREENSHOT

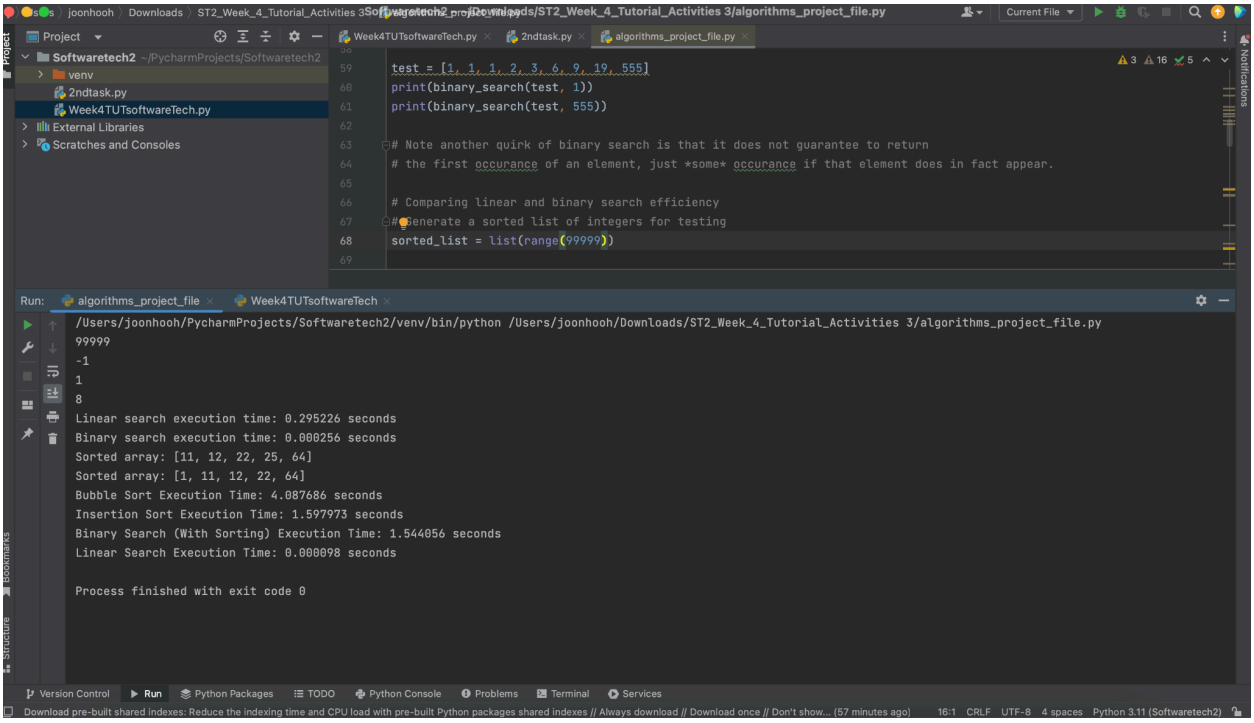


TASK 2:

Test1: value of 100



Test2: value of 99999



FOR LINEAR:

The two tests showed that the linear search time increased from 0.0002s to 0.2 seconds, showing a large difference of slower speed,

As time grows proportionally with the data size, the results show a big gap of how increased data size leads to the time also increasing.

FOR BINARY:

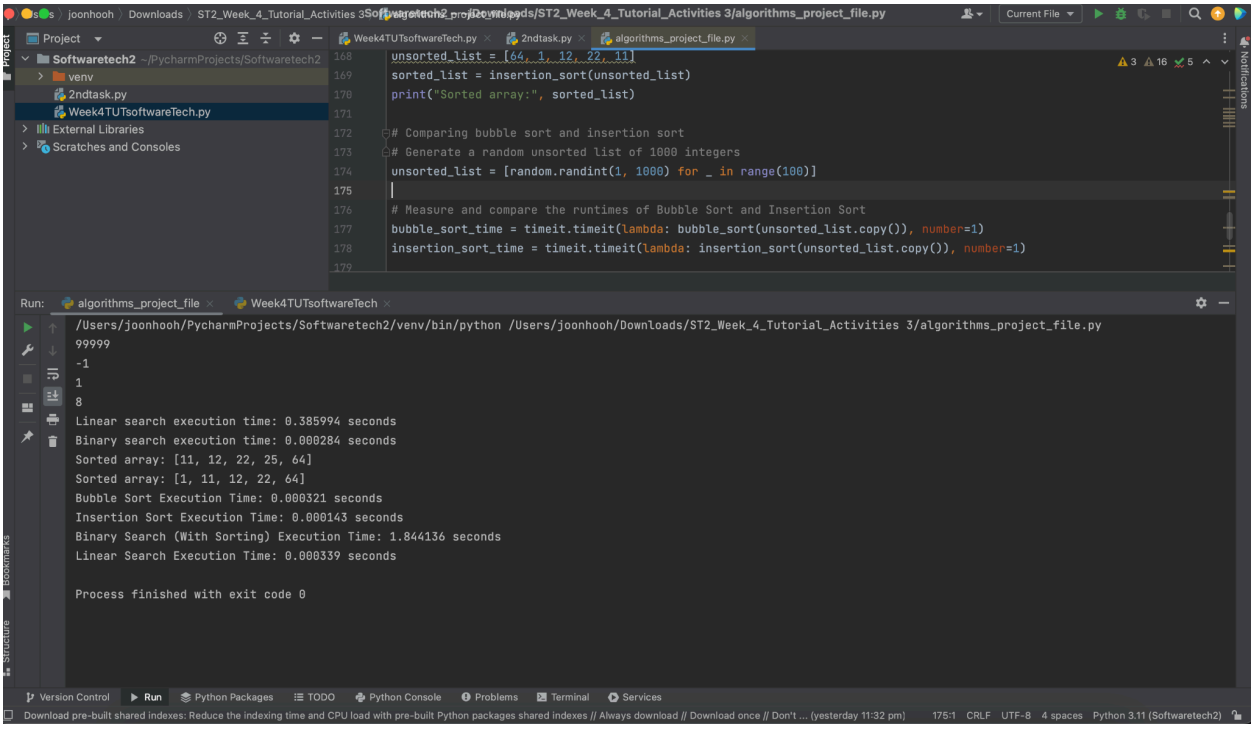
The two tests showed that the Binary Search increased only by a bit from 0.0001 to 0.00025, this is not as big as the linear search, however this demonstrates that time grows slowly with the data size.

COMPARING:

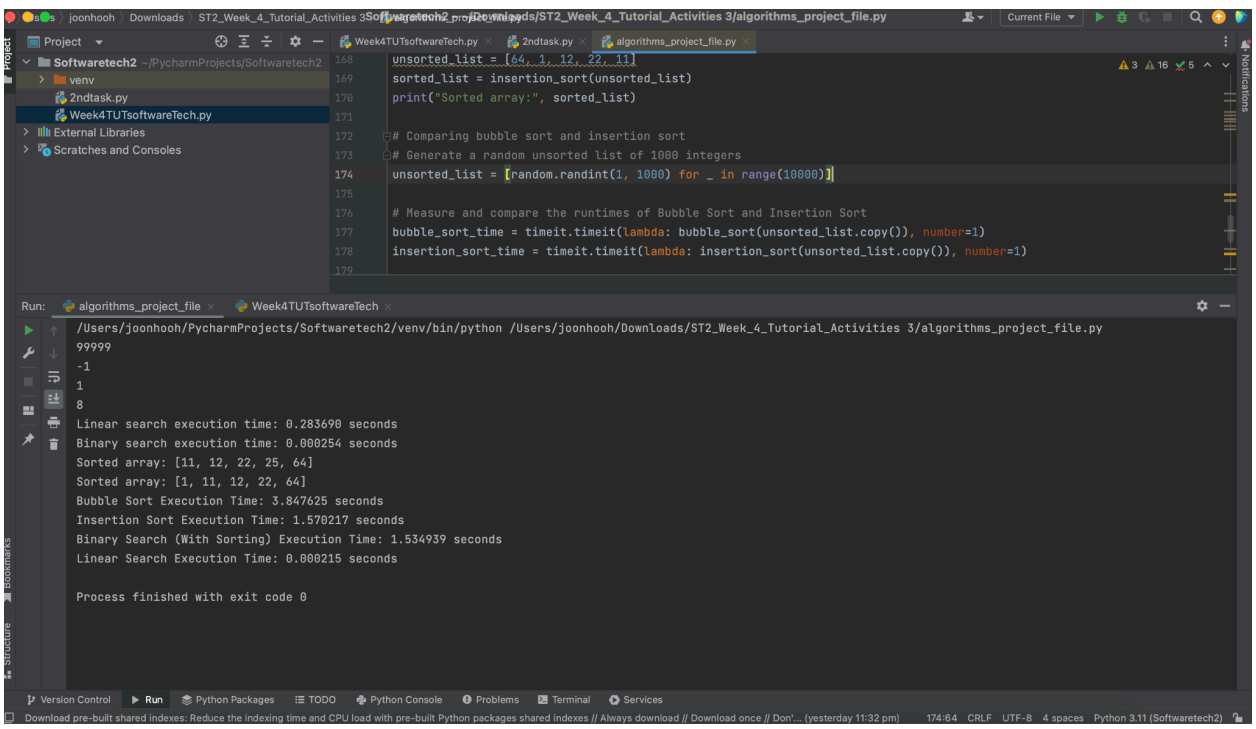
Binary seems to be the case of better performance as it shows that Binary search time grows logarithmically with the given data size, and linear search time grows proportionally with the data size. In conclusion it can be said that Binary is a lot faster than linear, especially when the data is larger (like how the 99999 data showed).

TASK3:

Value of 100:



Value of 10000:

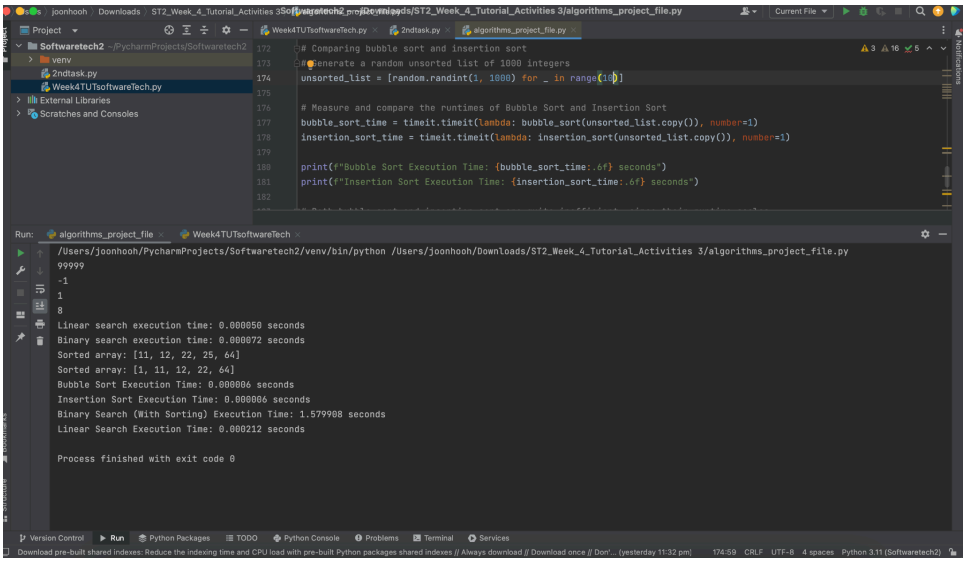


The insertion sort shows to have the upper hand over Bubble sort as it shows faster execution times, and the performance gap widens as the datasets go larger, especially shown in the 10000 input value as it saved 2.3 seconds.

TASK4:

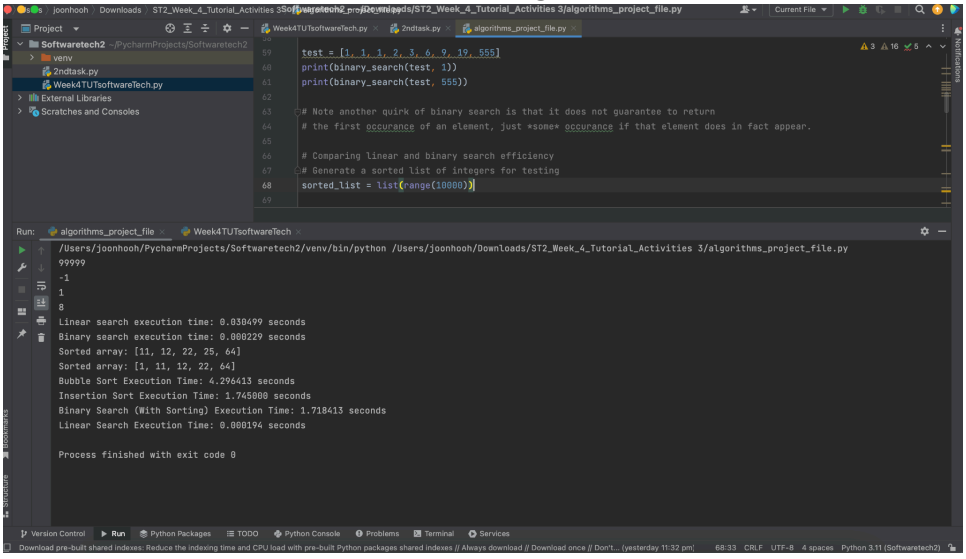
1st Combination:

Both unsorted_list and sorted_list range value of 10



Insertion sort time: 0.000006s
Linear search time: 0.000050s
Bubble sort time: 0.000006s
Binary search time: 0.000072s

Both unsorted_list and sorted_list range value of 10000



Insertion sort time: 1.7s
Linear search time: 0.03s
Bubble sort time: 4.2
Binary search time: 0.000229

Combination that performs best:
Insertion sort and binary search seems to show that it is the best combination.